

Render the shadow map to the depth buffer

Article • 10/20/2022

Render from the point of view of the light to create a two-dimensional depth map representing the shadow volume. The depth map masks the space that will be rendered in shadow. Part 2 of [Walkthrough: Implement shadow volumes using depth buffers in Direct3D 11](#).

Clear the depth buffer

Always clear the depth buffer before rendering to it.

C++

```
context->ClearRenderTargetView(m_deviceResources-  
>GetBackBufferRenderTargetView(), DirectX::Colors::CornflowerBlue);  
context->ClearDepthStencilView(m_shadowDepthView.Get(), D3D11_CLEAR_DEPTH |  
D3D11_CLEAR_STENCIL, 1.0f, 0);
```

Render the shadow map to the depth buffer

For the shadow rendering pass, specify a depth buffer but do not specify a render target.

Specify the light viewport, a vertex shader, and set the light space constant buffers. Use front face culling for this pass to optimize the depth values placed in the shadow buffer.

Note that on most devices, you can specify nullptr for the pixel shader (or skip specifying a pixel shader entirely). But some drivers may throw an exception when you call draw on the Direct3D device with a null pixel shader set. To avoid this exception, you can set a minimal pixel shader for the shadow rendering pass. The output of this shader is thrown away; it can call [discard](#) on every pixel.

Render the objects that can cast shadows, but don't bother rendering geometry that can't cast a shadow (like a floor in a room, or objects removed from the shadow pass for optimization reasons).

C++

```

void ShadowSceneRenderer::RenderShadowMap()
{
    auto context = m_deviceResources->GetD3DDeviceContext();

    // Render all the objects in the scene that can cast shadows onto
    themselves or onto other objects.

    // Only bind the ID3D11DepthStencilView for output.
    context->OMSetRenderTargets(
        0,
        nullptr,
        m_shadowDepthView.Get()
    );

    // Note that starting with the second frame, the previous call will
    display
    // warnings in VS debug output about forcing an unbind of the pixel
    shader
    // resource. This warning can be safely ignored when using shadow
    buffers
    // as demonstrated in this sample.

    // Set rendering state.
    context->RSSetState(m_shadowRenderState.Get());
    context->RSSetViewports(1, &m_shadowViewport);

    // Each vertex is one instance of the VertexPositionTexNormColor struct.
    UINT stride = sizeof(VertexPositionTexNormColor);
    UINT offset = 0;
    context->IASetVertexBuffers(
        0,
        1,
        m_vertexBuffer.GetAddressOf(),
        &stride,
        &offset
    );

    context->IASetIndexBuffer(
        m_indexBuffer.Get(),
        DXGI_FORMAT_R16_UINT, // Each index is one 16-bit unsigned integer
        (short).
        0
    );

    context->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
    context->IASetInputLayout(m_inputLayout.Get());

    // Attach our vertex shader.
    context->VSSetShader(
        m_simpleVertexShader.Get(),
        nullptr,
        0
    );

```

```

// Send the constant buffers to the Graphics device.
context->VSSetConstantBuffers(
    0,
    1,
    m_lightViewProjectionBuffer.GetAddressOf()
);

context->VSSetConstantBuffers(
    1,
    1,
    m_rotatedModelBuffer.GetAddressOf()
);

// In some configurations, it's possible to avoid setting a pixel shader
// (or set PS to nullptr). Not all drivers are tolerant of this, so to
be
// safe set a minimal shader here.
//
// Direct3D will discard output from this shader because the render
target
// view is unbound.
context->PSSetShader(
    m_textureShader.Get(),
    nullptr,
    0
);

// Draw the objects.
context->DrawIndexed(
    m_indexCountCube,
    0,
    0
);
}

```

Optimize the view frustum: Make sure your implementation computes a tight view frustum so that you get the most precision out of your depth buffer. See [Common Techniques to Improve Shadow Depth Maps](#) for more tips on shadow technique.

Vertex shader for shadow pass

Use a simplified version of your vertex shader to render just the vertex position in light space. Don't include any lighting normals, secondary transformations, and so on.

C++

```

PixelShaderInput main(VertexShaderInput input)
{
    PixelShaderInput output;
    float4 pos = float4(input.pos, 1.0f);
}

```

```
// Transform the vertex position into projected space.  
pos = mul(pos, model);  
pos = mul(pos, view);  
pos = mul(pos, projection);  
output.pos = pos;  
  
return output;  
}
```

In the next part of this walkthrough, learn how to add shadows by [rendering with depth testing](#).

Render the scene with depth testing

Article • 10/20/2022

Create a shadow effect by adding depth testing to your vertex (or geometry) shader and your pixel shader. Part 3 of [Walkthrough: Implement shadow volumes using depth buffers in Direct3D 11](#).

Include transformation for light frustum

Your vertex shader needs to compute the transformed light space position for each vertex. Provide the light space model, view, and projection matrices using a constant buffer. You can also use this constant buffer to provide the light position and normal for lighting calculations. The transformed position in light space will be used during the depth test.

C++

```
PixelShaderInput main(VertexShaderInput input)
{
    PixelShaderInput output;
    float4 pos = float4(input.pos, 1.0f);

    // Transform the vertex position into projected space.
    float4 modelPos = mul(pos, model);
    pos = mul(modelPos, view);
    pos = mul(pos, projection);
    output.pos = pos;

    // Transform the vertex position into projected space from the POV of
    the light.
    float4 lightSpacePos = mul(modelPos, lView);
    lightSpacePos = mul(lightSpacePos, lProjection);
    output.lightSpacePos = lightSpacePos;

    // Light ray
    float3 lRay = lPos.xyz - modelPos.xyz;
    output.lRay = lRay;

    // Camera ray
    output.view = eyePos.xyz - modelPos.xyz;

    // Transform the vertex normal into world space.
    float4 norm = float4(input.norm, 1.0f);
    norm = mul(norm, model);
    output.norm = norm.xyz;

    // Pass through the color and texture coordinates without modification.
    output.color = input.color;
```

```
    output.tex = input.tex;

    return output;
}
```

Next, the pixel shader will use the interpolated light space position provided by the vertex shader to test whether the pixel is in shadow.

Test whether the position is in the light frustum

First, check that the pixel is in the view frustum of the light by normalizing the X and Y coordinates. If they are both within the range [0, 1] then it's possible for the pixel to be in shadow. Otherwise you can skip the depth test. A shader can test for this quickly by calling [Saturate](#) and comparing the result against the original value.

C++

```
// Compute texture coordinates for the current point's location on the
// shadow map.
float2 shadowTexCoords;
shadowTexCoords.x = 0.5f + (input.lightSpacePos.x / input.lightSpacePos.w *
0.5f);
shadowTexCoords.y = 0.5f - (input.lightSpacePos.y / input.lightSpacePos.w *
0.5f);
float pixelDepth = input.lightSpacePos.z / input.lightSpacePos.w;

float lighting = 1;

// Check if the pixel texture coordinate is in the view frustum of the
// light before doing any shadow work.
if ((saturate(shadowTexCoords.x) == shadowTexCoords.x) &&
    (saturate(shadowTexCoords.y) == shadowTexCoords.y) &&
    (pixelDepth > 0))
{
```

Depth test against the shadow map

Use a sample comparison function (either [SampleCmp](#) or [SampleCmpLevelZero](#)) to test the pixel's depth in light space against the depth map. Compute the normalized light space depth value, which is z / w , and pass the value to the comparison function. Since we use a LessOrEqual comparison test for the sampler, the intrinsic function returns zero when the comparison test passes; this indicates that the pixel is in shadow.

C++

```

// Use an offset value to mitigate shadow artifacts due to imprecise
// floating-point values (shadow acne).
//
// This is an approximation of epsilon * tan(acos(saturate(NdotL))):
float margin = acos(saturate(NdotL));
#ifdef LINEAR
// The offset can be slightly smaller with smoother shadow edges.
float epsilon = 0.0005 / margin;
#else
float epsilon = 0.001 / margin;
#endif
// Clamp epsilon to a fixed range so it doesn't go overboard.
epsilon = clamp(epsilon, 0, 0.1);

// Use the SampleCmpLevelZero Texture2D method (or SampleCmp) to sample from
// the shadow map, just as you would with Direct3D feature level 10_0 and
// higher. Feature level 9_1 only supports LessOrEqual, which returns 0 if
// the pixel is in the shadow.
lighting = float(shadowMap.SampleCmpLevelZero(
    shadowSampler,
    shadowTexCoords,
    pixelDepth + epsilon
));

```

Compute lighting in or out of shadow

If the pixel is not in shadow, the pixel shader should compute direct lighting and add it to the pixel value.

C++

```

return float4(input.color * (ambient + DplusS(N, L, NdotL, input.view)),
1.f);

```

C++

```

float3 DplusS(float3 N, float3 L, float NdotL, float3 view)
{
    const float3 Kdiffuse = float3(.5f, .5f, .4f);
    const float3 Kspecular = float3(.2f, .2f, .3f);
    const float exponent = 3.f;

    // Compute the diffuse coefficient.
    float diffuseConst = saturate(NdotL);

    // Compute the diffuse lighting value.
    float3 diffuse = Kdiffuse * diffuseConst;

```



```
// Compute the specular highlight.  
float3 R = reflect(-L, N);  
float3 V = normalize(view);  
float3 RdotV = dot(R, V);  
float3 specular = Kspecular * pow(saturate(RdotV), exponent);  
  
return (diffuse + specular);  
}
```

Otherwise, the pixel shader should compute the pixel value using ambient lighting.

C++

```
return float4(input.color * ambient, 1.f);
```

In the next part of this walkthrough, learn how to [Support shadow maps on a range of hardware](#).

Support shadow maps on a range of hardware

Article • 10/20/2022

Render higher-fidelity shadows on faster devices and faster shadows on less powerful devices. Part 4 of [Walkthrough: Implement shadow volumes using depth buffers in Direct3D 11](#).

Comparison filter types

Only use linear filtering if the device can afford the performance penalty. Generally, Direct3D feature level 9_1 devices don't have enough power to spare for linear filtering on shadows. Use point filtering instead on these devices. When you use linear filtering, adjust the pixel shader so that it blends the shadow edges.

Create the comparison sampler for point filtering:

C++

```
D3D11_SAMPLER_DESC comparisonSamplerDesc;
ZeroMemory(&comparisonSamplerDesc, sizeof(D3D11_SAMPLER_DESC));
comparisonSamplerDesc.AddressU = D3D11_TEXTURE_ADDRESS_BORDER;
comparisonSamplerDesc.AddressV = D3D11_TEXTURE_ADDRESS_BORDER;
comparisonSamplerDesc.AddressW = D3D11_TEXTURE_ADDRESS_BORDER;
comparisonSamplerDesc.BorderColor[0] = 1.0f;
comparisonSamplerDesc.BorderColor[1] = 1.0f;
comparisonSamplerDesc.BorderColor[2] = 1.0f;
comparisonSamplerDesc.BorderColor[3] = 1.0f;
comparisonSamplerDesc.MinLOD = 0.f;
comparisonSamplerDesc.MaxLOD = D3D11_FLOAT32_MAX;
comparisonSamplerDesc.MipLODBias = 0.f;
comparisonSamplerDesc.MaxAnisotropy = 0;
comparisonSamplerDesc.ComparisonFunc = D3D11_COMPARISON_LESS_EQUAL;
comparisonSamplerDesc.Filter = D3D11_FILTER_COMPARISON_MIN_MAG_MIP_POINT;

// Point filtered shadows can be faster, and may be a good choice when
// rendering on hardware with lower feature levels. This sample has a
// UI option to enable/disable filtering so you can see the difference
// in quality and speed.

DX::ThrowIfFailed(
    pD3DDevice->CreateSamplerState(
        &comparisonSamplerDesc,
        &m_comparisonSampler_point
    )
);
```

Then create a sampler for linear filtering:

C++

```
comparisonSamplerDesc.Filter = D3D11_FILTER_COMPARISON_MIN_MAG_MIP_LINEAR;
DX::ThrowIfFailed(
    pD3DDevice->CreateSamplerState(
        &comparisonSamplerDesc,
        &m_comparisonSampler_linear
    )
);
```

Choose a sampler:

C++

```
ID3D11PixelShader* pixelShader;
ID3D11SamplerState** comparisonSampler;
if (m_useLinear)
{
    pixelShader = m_shadowPixelShader_linear.Get();
    comparisonSampler = m_comparisonSampler_linear.GetAddressOf();
}
else
{
    pixelShader = m_shadowPixelShader_point.Get();
    comparisonSampler = m_comparisonSampler_point.GetAddressOf();
}

// Attach our pixel shader.
context->PSSetShader(
    pixelShader,
    nullptr,
    0
);

context->PSSetSamplers(0, 1, comparisonSampler);
context->PSSetShaderResources(0, 1, m_shadowResourceView.GetAddressOf());
```

Blend shadow edges with linear filtering:

C++

```
// Blends the shadow area into the lit area.
float3 light = lighting * (ambient + DplusS(N, L, NdotL, input.view));
float3 shadow = (1.0f - lighting) * ambient;
return float4(input.color * (light + shadow), 1.f);
```

Shadow buffer size

Larger shadow maps won't look as blocky but they take up more space in graphics memory. Experiment with different shadow map sizes in your game and observe the results in different types of devices and different display sizes. Consider an optimization like cascaded shadow maps to get better results with less graphics memory. See [Common Techniques to Improve Shadow Depth Maps](#).

Shadow buffer depth

Greater precision in the shadow buffer will give more accurate depth test results, which helps avoid issues like z-buffer fighting. But like larger shadow maps, greater precision takes up more memory. Experiment with different depth precision types in your game - DXGI_FORMAT_R24G8_TYPELESS versus DXGI_FORMAT_R16_TYPELESS - and observe the speed and quality on different feature levels.

Optimizing precompiled shaders

Universal Windows Platform (UWP) apps can use dynamic shader compilation, but it's faster to use dynamic shader linking. You can also use compiler directives and `#ifdef` blocks to create different versions of shaders. This is done by opening the Visual Studio project file in a text editor and adding multiple `<FxCompiler>` entries for the HLSL (each with the appropriate preprocessor definitions). Note that this necessitates different filenames; in this case, Visual Studio appends `_point` and `_linear` to the different versions of the shader.

The project file entry for the linear filtered version of the shader defines `LINEAR`:

XML

```
<FxCompile Include="Content\ShadowPixelShader.hlsl">
  <ShaderType
    Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">Pixel</ShaderType>
  <ShaderType
    Condition="'$(Configuration)|$(Platform)'=='Release|ARM'">Pixel</ShaderType>
  <ShaderType
    Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">Pixel</ShaderType>
  <ShaderType
    Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">Pixel</ShaderType>
  <ShaderType
    Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">Pixel</ShaderType>
  <ShaderType
    Condition="'$(Configuration)|$(Platform)'=='Release|x64'">Pixel</ShaderType>
```

```

    <DisableOptimizations
Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">>false</DisableOptimizations>
    <EnableDebuggingInformation
Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">>true</EnableDebuggingInformation>
    <DisableOptimizations
Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">>false</DisableOptimizations>
    <DisableOptimizations
Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">>false</DisableOptimizations>
    <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">$(OutDir)%(Filename)_linear.cso</ObjectFileOutput>
    <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Release|ARM'">$(OutDir)%(Filename)_linear.cso</ObjectFileOutput>
    <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">$(OutDir)%(Filename)_linear.cso</ObjectFileOutput>
    <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">$(OutDir)%(Filename)_linear.cso</ObjectFileOutput>
    <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">$(OutDir)%(Filename)_linear.cso</ObjectFileOutput>
    <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Release|x64'">$(OutDir)%(Filename)_linear.cso</ObjectFileOutput>
    <PreprocessorDefinitions
Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">LINEAR</PreprocessorDefinitions>
    <PreprocessorDefinitions
Condition="'$(Configuration)|$(Platform)'=='Release|ARM'">LINEAR</PreprocessorDefinitions>
    <PreprocessorDefinitions
Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">LINEAR</PreprocessorDefinitions>
    <PreprocessorDefinitions
Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">LINEAR</PreprocessorDefinitions>
    <PreprocessorDefinitions
Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">LINEAR</PreprocessorDefinitions>
    <PreprocessorDefinitions
Condition="'$(Configuration)|$(Platform)'=='Release|x64'">LINEAR</PreprocessorDefinitions>
</FxCompile>

```

The project file entry for the linear filtered version of the shader does not include preprocessor definitions:

```

<FxCompile Include="Content\ShadowPixelShader.hlsl">
  <ShaderType
Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">Pixel</ShaderType>
  <ShaderType
Condition="'$(Configuration)|$(Platform)'=='Release|ARM'">Pixel</ShaderType>
  <ShaderType
Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">Pixel</ShaderType>
  <ShaderType
Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">Pixel</ShaderType>
  <ShaderType
Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">Pixel</ShaderType>
  <ShaderType
Condition="'$(Configuration)|$(Platform)'=='Release|x64'">Pixel</ShaderType>
  <DisableOptimizations
Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">>false</DisableOptimizations>
  <EnableDebuggingInformation
Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">>true</EnableDebuggingInformation>
  <DisableOptimizations
Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">>false</DisableOptimizations>
  <DisableOptimizations
Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">>false</DisableOptimizations>
  <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Debug|ARM'">$(OutDir)%(Filename)_point.cso</ObjectFileOutput>
  <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Release|ARM'">$(OutDir)%(Filename)_point.cso</ObjectFileOutput>
  <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Debug|Win32'">$(OutDir)%(Filename)_point.cso</ObjectFileOutput>
  <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Release|Win32'">$(OutDir)%(Filename)_point.cso</ObjectFileOutput>
  <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Debug|x64'">$(OutDir)%(Filename)_point.cso</ObjectFileOutput>
  <ObjectFileOutput
Condition="'$(Configuration)|$(Platform)'=='Release|x64'">$(OutDir)%(Filename)_point.cso</ObjectFileOutput>
</FxCompile>

```

DirectX 11 porting FAQ

FAQ

Answers to frequently-asked questions about porting games to Universal Windows Platform (UWP).

Is porting my game going to be a set of search-and-replace operations on API methods, or do I need to plan for a more thoughtful porting process?

Direct3D 11 is a significant upgrade from Direct3D 9. There are several paradigm shifts, including separate APIs for the virtualized graphics adapter and its context as well as a new layer of polymorphism for device resources. Your game can still use graphics hardware in essentially the same way, but you'll need to learn about the new Direct3D 11 API architecture and update each part of your graphics code to use the correct API components. See [Porting concepts and considerations](#).

What is the new device context for? Am I supposed to replace my Direct3D 9 device with the Direct3D 11 device, the device context, or both?

The Direct3D device is now used to create resources in video memory, while the device context is used to set pipeline state and generate rendering commands. For more info see: [What are the most important changes since Direct3D 9?](#)

Do I have to update my game timer for UWP?

[QueryPerformanceCounter](#), along with [QueryPerformanceFrequency](#), is still the best way to implement a game timer for UWP apps.

You should be aware of a nuance with timers and the UWP app lifecycle.

Suspend/resume is different from a player re-launching a desktop game because your game will resume a snapshot in time from when the game was last played. If a large amount of time has passed - for example, a few weeks - some game timer implementations might not behave gracefully. You can use app lifecycle events to reset your timer when the game resumes.

Games that still use the RDTSC instruction need to upgrade. See [Game Timing and Multicore Processors](#).

My game code is based on D3DX and DXUT. Is there anything available that can help me migrate my code?

The [DirectX Tool Kit \(DirectXTK\)](#) community project offers helper classes for use with Direct3D 11.

How do I maintain code paths for the desktop and the Microsoft Store?

Chuck Walbourn's article series titled [Dual-use Coding Techniques for Games](#) offers guidance on sharing code between the desktop and the Microsoft Store code paths.

How do I load image resources in my DirectX UWP app?

There are two API paths for loading images:

- The content pipeline converts images into DDS files used as Direct3D texture resources. See [Using 3-D Assets in Your Game or App](#).
- The [Windows Imaging Component](#) can be used to load images from a variety of formats, and can be used for Direct2D bitmaps as well as Direct3D texture resources.

You can also use the DDSTextureLoader, and the WICTextureLoader, from the [DirectXTK](#) or [DirectXTex](#).

Where is the DirectX SDK?

The DirectX SDK is included as part of the Windows SDK. The most recent DirectX SDK that was separate from the Windows SDK was in June 2010. Direct3D samples are in the Code Gallery along with the rest of the Windows app samples.

What about DirectX redistributables?

The vast majority of components in the Windows SDK are already included in supported versions of the OS, or have no DLL component (such as DirectXMath). All Direct3D API components that UWP apps can use will already be available to your game; you don't need to redistribute them.

Win32 desktop applications still use DirectSetup, so if you are also upgrading the desktop version of your game see [Direct3D 11 Deployment for Game Developers](#).

Is there any way I can update my desktop code to DirectX 11 before moving away from Effects?

See the [Effects for Direct3D 11 Update](#). Effects 11 helps remove dependencies on legacy DirectX SDK headers; it's intended for use as a porting aid and can only be used with desktop apps.

Is there a path for porting my DirectX 8 game to UWP?

Yes:

- Read [Converting to Direct3D 9](#).
- Make sure your game has no remnants of the fixed pipeline - see [Deprecated Features](#).
- Then take the DirectX 9 porting path: [Port from D3D 9 to UWP](#).

Can I port my DirectX 10 or 11 game to UWP?

DirectX 10.x and 11 desktop games are easy to port to UWP. See [Migrating to Direct3D 11](#).

How do I choose the right display device in a multi-monitor system?

The user selects which monitor your app is displayed on. Let Windows provide the correct adapter by calling [D3D11CreateDevice](#) with the first parameter set to `nullptr`. Then get the device's [IDXGIDevice interface](#), call [GetAdapter](#) and use the DXGI adapter to create the swap chain.

How do I turn on antialiasing?

Antialiasing (multisampling) is enabled when you create the Direct3D device. Enumerate multisampling support by calling [CheckMultisampleQualityLevels](#), then set multisample options in the [DXGI_SAMPLE_DESC structure](#) when you call [CreateSurface](#).

My game renders using multithreading and/or deferred rendering. What do I need to know for Direct3D 11?

Visit [Introduction to Multithreading in Direct3D 11](#) to get started. For a list of key differences, see [Threading Differences between Direct3D Versions](#). Note that deferred rendering uses a device *deferred context* instead of an *immediate context*.

Where can I read more about the programmable pipeline since Direct3D 9?

Visit the following topics:

- [Programming Guide for HLSL](#)
- [Direct3D 10 Frequently Asked Questions](#)

What should I use instead of the .x file format for my models?

While we don't have an official replacement for the .x file format, many of the samples utilize the SDKMesh format. Visual Studio also has a [content pipeline](#) that compiles several popular formats into CMO files that can be loaded with code from the Visual Studio 3D starter kit, or loaded using the [DirectXTK](#) [↗](#).

How do I debug my shaders?

Microsoft Visual Studio includes diagnostic tools for DirectX graphics. See [Debugging DirectX Graphics](#).

What is the Direct3D 11 equivalent for x function?

See the [function mapping](#) provided in Map DirectX 9 features to DirectX 11 APIs.

What is the DXGI_FORMAT equivalent of y surface format?

See the [surface format mapping](#) provided in Map DirectX 9 features to DirectX 11 APIs.

Feedback

Was this page helpful?

[Provide product feedback](#) [↗](#) | [Get help at Microsoft Q&A](#)

Port from OpenGL ES 2.0 to Direct3D 11

Article • 10/20/2022

Includes articles, overviews, and walkthroughs for porting an OpenGL ES 2.0 graphics pipeline to a Direct3D 11 and the Windows Runtime.

Note An intermediate step to porting your OpenGL ES 2.0 project is to use ANGLE for Microsoft Store. ANGLE allows you to run OpenGL ES content on Windows by translating OpenGL ES API calls to DirectX 11 API calls. For more information about ANGLE, go to the [ANGLE for Microsoft Store Wiki](#).

Topic	Description
Map OpenGL ES 2.0 to Direct3D 11.1	When starting the process of porting your graphics architecture from OpenGL ES 2.0 to Direct3D for the first time, familiarize yourself with the key differences between the APIs. The topics in this section help you plan your port strategy and the API changes that you must make when moving your graphics processing to Direct3D.
How to: port a simple OpenGL ES 2.0 renderer to Direct3D 11.1	For this porting exercise, we'll start with the basics: bringing a simple renderer for a spinning, vertex-shaded cube from OpenGL ES 2.0 into Direct3D, such that it matches the DirectX 11 App (Universal Windows) template from Visual Studio 2015.
OpenGL ES 2.0 to Direct3D 11.1 reference	Use these reference topics to look up API mapping and short code samples when porting from OpenGL ES 2.0 to Direct3D 11.

Map OpenGL ES 2.0 to Direct3D 11

Article • 10/20/2022

When starting the process of porting your graphics architecture from OpenGL ES 2.0 to Direct3D for the first time, familiarize yourself with the key differences between the APIs. The topics in this section help you plan your port strategy and the API changes that you must make when moving your graphics processing to Direct3D.

Topic	Description
Plan your port from OpenGL ES 2.0 to Direct3D	If you are porting a game from the iOS or Android platforms, you have probably made a significant investment in OpenGL ES 2.0. When preparing to move your graphics pipeline codebase to Direct3D 11 and the Windows Runtime, there are a few things you should consider before you start.
Compare EGL code to DXGI and Direct3D	The DirectX Graphics Interface (DXGI) and several Direct3D APIs serve the same role as EGL. This topic helps you understand DXGI and Direct3D 11 from the perspective of EGL.
Compare OpenGL ES 2.0 buffers, uniforms, and vertex attributes to Direct3D	During the process of porting to Direct3D 11 from OpenGL ES 2.0, you must change the syntax and API behavior for passing data between the app and the shader programs.
Compare the OpenGL ES 2.0 shader pipeline to Direct3D	Conceptually, the Direct3D 11 shader pipeline is very similar to the one in OpenGL ES 2.0. In terms of API design, however, the major components for creating and managing the shader stages are parts of two primary interfaces, ID3D11Device1 and ID3D11DeviceContext1 . This topic attempts to map common OpenGL ES 2.0 shader pipeline API patterns to the Direct3D 11 equivalents in these interfaces.

Notes on specific OpenGL ES 2.0 providers

These topics use the Khronos OpenGL ES 2.0 specification with platform-agnostic C. Both iOS and Android utilize the same specification and OpenGL ES 2.0 code developed

for those platforms is very similar to the code snippets we will walk through, although they are typically exposed as object-oriented APIs. Also, due to the intricacies and language differences of each platform, there may be minor differences, especially in method parameter types, or in general language syntax. iOS, for instance, uses Objective-C. Android has the capability to use C++; however, some developers may have relied on a pure Java implementation. With that in mind, these topics should still be useful as the overall concepts, structure and usage of the OpenGL ES APIs do not differ.

Plan your port from OpenGL ES 2.0 to Direct3D

Article • 10/20/2022

Important APIs

- [Direct3D 11](#)
- [Visual C++](#)

If you are porting a game from the iOS or Android platforms, you have probably made a significant investment in OpenGL ES 2.0. When preparing to move your graphics pipeline codebase to Direct3D 11 and the Windows Runtime, there are a few things you should consider before you start.

Most porting efforts usually involving initially walking the codebase and mapping common APIs and patterns between the two models. You'll find this process a bit easier if you take some time to read and review this topic.

Here are some things to be aware of when porting graphics from OpenGL ES 2.0 to Direct3D 11.

Notes on specific OpenGL ES 2.0 providers

The porting topics in this section reference the Windows implementation of the OpenGL ES 2.0 specification created by the Khronos Group. All OpenGL ES 2.0 code samples were developed using Visual Studio 2012 and basic Windows C syntax. If you are coming from an Objective-C (iOS) or Java (Android) codebase, be aware that the provided OpenGL ES 2.0 code samples may not use similar API calling syntax or parameters. This guidance tries to stay as platform agnostic as possible.

This documentation only uses the 2.0 specification APIs for the OpenGL ES code and reference. If you are porting from OpenGL ES 1.1 or 3.0, this content can still prove useful, although some of the OpenGL ES 2.0 code examples and context may be unfamiliar.

The Direct3D 11 samples in these topics use Microsoft Windows C++ with Component Extensions (CX). For more info on this version of the C++ syntax, read [Visual C++](#), [Component Extensions for Runtime Platforms](#), and [Quick Reference \(C++\CX\)](#).

Understand your hardware requirements and resources

The set of graphics processing features supported by OpenGL ES 2.0 roughly maps to the features provided in Direct3D 9.1. If you want to take advantage of the more advanced features provided in Direct3D 11, review the [Direct3D 11](#) documentation when planning your port, or review the [Port from DirectX 9 to Universal Windows Platform \(UWP\)](#) topics when you're done with the initial effort.

To make your initial porting effort simple, start with a Visual Studio Direct3D template. It provides a basic renderer already configured for you, and supports UWP app features like recreating resources on window changes and Direct3D feature levels.

Understand Direct3D feature levels

Direct3D 11 provides support for hardware "feature levels" from 9_1 (Direct3D 9.1) for 11_1. These feature levels indicate the availability of certain graphics features and resources. Typically, most OpenGL ES 2.0 platforms support a Direct3D 9.1 (feature level 9_1) set of features.

Review DirectX graphics features and APIs

 Expand table

API Family	Description
DXGI	The DirectX Graphics Infrastructure (DXGI) provides an interface between the graphics hardware and Direct3D. It sets the device adapter and hardware configuration using the IDXGIAdapter and IDXGIDevice1 COM interfaces. Use it to create and configure your buffers and other window resources. Notably, the IDXGIFactory2 factory pattern is used to acquire the graphics resources, including the swap chain (a set of frame buffers). Since DXGI owns the swap chain, the IDXGISwapChain1 interface is used to present frames to the screen.
Direct3D	Direct3D is the set of APIs that provide a virtual representation of the graphics interface and allow you to draw graphics using it. Version 11, is roughly comparable, feature-wise, to OpenGL 4.3. (OpenGL ES 2.0, on the other hand, is similar to DirectX9, feature-wise, and OpenGL 2.0, but with OpenGL 3.0's unified shader pipeline.) Most of the heavy lifting is done with the ID3D11Device1 and ID3D11DeviceContext1 interfaces which provide access to individual resources and subresources, and the rendering context, respectively.
Direct2D	Direct2D provides a set of APIs for GPU-accelerated 2D rendering. It can be

API Family	Description
	considered similar in purpose to OpenVG.
DirectWrite	DirectWrite provides a set of APIs for GPU-accelerated, high-quality font rendering.
DirectXMath	DirectXMath provides a set of APIs and macros for handling common linear algebra and trigonometric types, values, and functions. These types and functions are designed to work well with Direct3D and its shader operations.
DirectX HLSL	The current HLSL syntax used by Direct3D shaders. It implements Direct3D Shader Model 5.0.

Review the Windows Runtime APIs and template library

The Windows Runtime APIs provide the overall infrastructure for UWP apps. Review them [here](#).

Key Windows Runtime APIs used in porting your graphics pipeline include:

- [Windows::UI::Core::CoreWindow](#)
- [Windows::UI::Core::CoreDispatcher](#)
- [Windows::ApplicationModel::Core::IFrameworkView](#)
- [Windows::ApplicationModel::Core::CoreApplicationView](#)

Additionally, the Windows Runtime C++ Template Library (WRL) is a template library that provides a low-level way to author and use Windows Runtime components. The Direct3D 11 APIs for UWP apps are best used in conjunctions with the interfaces and types in this library, such as smart pointers ([ComPtr](#)). For more info on the WRL, read [Windows Runtime C++ Template Library \(WRL\)](#).

Change your coordinate system

One difference that sometimes confuses early port efforts is the change from OpenGL's traditional right-handed coordinate system to Direct3D's default left-handed coordinate system. This change in coordinate modeling affects many parts of your game, from the setup and configuration of your vertex buffers to many of your matrix math functions. The two most important changes to make are:

- Flip the order of triangle vertices so that Direct3D traverses them clockwise from the front. For example, if your vertices are indexed as 0, 1, and 2 in your OpenGL

pipeline, pass them to Direct3D as 0, 2, 1 instead.

- Use the view matrix to scale world space by -1.0f in the z direction, effectively reversing the z-axis coordinates. To do this, flip the sign of the values at positions M31, M32, and M33 in your view matrix (when porting it to the **Matrix** type). If M34 is not 0, flip its sign as well.

However, Direct3D can support a right-handed coordinate system. DirectXMath provides a number of functions that operate on and across both left-handed and right-handed coordinate systems. They can be used to preserve some of your original mesh data and matrix processing. They include:

 Expand table

DirectXMath matrix function	Description
XMMatrixLookAtLH	Builds a view matrix for a left-handed coordinate system using a camera position, an up direction, and a focal point.
XMMatrixLookAtRH	Builds a view matrix for a right-handed coordinate system using a camera position, an up direction, and a focal point.
XMMatrixLookToLH	Builds a view matrix for a left-handed coordinate system using a camera position, an up direction, and a camera direction.
XMMatrixLookToRH	Builds a view matrix for a right-handed coordinate system using a camera position, an up direction, and a camera direction.
XMMatrixOrthographicLH	Builds an orthogonal projection matrix for a left-handed coordinate system.
XMMatrixOrthographicOffCenterLH	Builds a custom orthogonal projection matrix for a left-handed coordinate system.
XMMatrixOrthographicOffCenterRH	Builds a custom orthogonal projection matrix for a right-handed coordinate system.
XMMatrixOrthographicRH	Builds an orthogonal projection matrix for a right-handed coordinate system.
XMMatrixPerspectiveFovLH	Builds a left-handed perspective projection matrix based on a field of view.
XMMatrixPerspectiveFovRH	Builds a right-handed perspective projection matrix based on a field of view.

DirectXMath matrix function	Description
XMMatrixPerspectiveLH	Builds a left-handed perspective projection matrix.
XMMatrixPerspectiveOffCenterLH	Builds a custom version of a left-handed perspective projection matrix.
XMMatrixPerspectiveOffCenterRH	Builds a custom version of a right-handed perspective projection matrix.
XMMatrixPerspectiveRH	Builds a right-handed perspective projection matrix.

OpenGL ES2.0-to-Direct3D 11 porting Frequently Asked Questions

- Question: "In general, can I search for certain strings or patterns in my OpenGL code and replace them with the Direct3D equivalents?"
- Answer: No. OpenGL ES 2.0 and Direct3D 11 come from different generations of graphics pipeline modeling. While there are some surface similarities between concepts and APIs, such as the rendering context and the instancing of shaders, you should review this guidance as well as the Direct3D 11 reference so you can make the best choices when recreating your pipeline instead of attempting a 1-to-1 mapping. However, if you are porting from GLSL to HLSL, creating a set of common aliases for GLSL variables, intrinsics, and functions can not only make porting easier, it allows you to maintain only one set of shader code files.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Compare EGL code to DXGI and Direct3D

Article • 10/20/2022

Important APIs

- [ID3D11Device1](#)
- [ID3D11DeviceContext1](#)
- [CoreWindow](#)

The DirectX Graphics Interface (DXGI) and several Direct3D APIs serve the same role as EGL. This topic helps you understand DXGI and Direct3D 11 from the perspective of EGL.

DXGI and Direct3D, like EGL, provide methods to configure graphics resources, obtain a rendering context for your shaders to draw into, and to display the results in a window. However, DXGI and Direct3D have quite a few more options, and require more effort to set up correctly when porting from EGL.

Note This guidance is based off the Khronos Group's open specification for EGL 1.4, found here: [Khronos Native Platform Graphics Interface \(EGL Version 1.4 - April 6, 2011\) \[PDF\]](#) [↗](#). Differences in syntax specific to other platforms and development languages are not covered in this guidance.

How does DXGI and Direct3D compare?

The big advantage of EGL over DXGI and Direct3D is that it is relatively simple to start drawing to a window surface. This is because OpenGL ES 2.0—and therefore EGL—is a specification implemented by multiple platform providers, whereas DXGI and Direct3D are a single reference that hardware vendor drivers must conform to. This means that Microsoft must implement a set of APIs that enable the broadest possible set of vendor features, rather than focusing on a functional subset offered by a specific vendor, or by combining vendor-specific setup commands into simpler APIs. On the other hand, Direct3D provides a single set of APIs that cover a very broad range of graphics hardware platforms and feature levels, and offer more flexibility for developers experienced with the platform.

Like EGL, DXGI and Direct3D provide APIs for the following behaviors:

- Obtaining, and reading and writing to a frame buffer (called a "swap chain" in DXGI).
- Associating the frame buffer with a UI window.
- Obtaining and configuring rendering contexts in which to draw.
- Issuing commands to the graphics pipeline for a specific rendering context.
- Creating and managing shader resources, and associating them with a rendering content.
- Rendering to specific render targets (such as textures).
- Updating the window's display surface with the results of rendering with the graphics resources.

To see the basic Direct3D process for configuring the graphics pipeline, check out the DirectX 11 App (Universal Windows) template in Microsoft Visual Studio 2015. The base rendering class in it provides a good baseline for setting up the Direct3D 11 graphics infrastructure and configuring basic resources on it, as well as supporting Universal Windows Platform (UWP) app features such as screen rotation.

EGL has very few APIs relative to Direct3D 11, and navigating the latter can be a challenge if you aren't familiar with the naming and jargon particular to the platform. Here's a simple overview to help you get oriented.

First, review the basic EGL object to Direct3D interface mapping:

 Expand table

EGL abstraction	Similar Direct3D representation
EGLDisplay	In Direct3D (for UWP apps), the display handle is obtained through the Windows::UI::CoreWindow API (or the ICoreWindowInterop interface that exposes the HWND). The adapter and hardware configuration are set with the IDXGIAdapter and IDXGIDevice1 COM interfaces, respectively.
EGLSurface	In Direct3D, the buffers and other window resources (visible or offscreen) are created and configured by specific DXGI interfaces, including IDXGIFactory2 (a factory pattern implementation used to acquire DXGI resources such as the IDXGISwapChain1 (display buffers). The ID3D11Device1 that represents the graphics device and its resources, is acquired with D3D11Device::CreateDevice . For render targets, use the ID3D11RenderTargetView interface.
EGLContext	In Direct3D, you configure and issue commands to the graphics pipeline with the ID3D11DeviceContext1 interface.
EGLConfig	In Direct3D 11, you create and configure graphics resources such as a buffers, textures, stencils and shaders with methods on the ID3D11Device1 interface.

Now, here's the most basic process for setting up a simple graphics display, resources and context in DXGI and Direct3D for a UWP app.

1. Obtain a handle to the [CoreWindow](#) object for the app's core UI thread by calling [CoreWindow::GetForCurrentThread](#).
2. For UWP apps, acquire a swap chain from the [IDXGIAdapter2](#) with [IDXGIFactory2::CreateSwapChainForCoreWindow](#), and pass it the [CoreWindow](#) reference you obtained in step 1. You will get an [IDXGISwapChain1](#) instance in return. Scope it to your renderer object and its rendering thread.
3. Obtain [ID3D11Device1](#) and [ID3D11DeviceContext1](#) instances by calling the [D3D11Device::CreateDevice](#) method. Scope them to your renderer object as well.
4. Create shaders, textures, and other resources using methods on your renderer's [ID3D11Device1](#) object.
5. Define buffers, run shaders and manage the pipeline stages using methods on your renderer's [ID3D11DeviceContext1](#) object.
6. When the pipeline has executed and a frame is drawn to the back buffer, present it to the screen with [IDXGISwapChain1::Present1](#).

To examine this process in more detail, review [Getting started with DirectX graphics](#). The rest of this article covers many of the common steps for basic graphics pipeline setup and management.

Note Windows Desktop apps have different APIs for obtaining a Direct3D swap chain, such as [D3D11Device::CreateDeviceAndSwapChain](#), and do not use a [CoreWindow](#) object.

Obtaining a window for display

In this example, `eglGetDisplay` is passed an `HWND` for a window resource specific to the Microsoft Windows platform. Other platforms, such as Apple's iOS (Cocoa) and Google's Android, have different handles or references to window resources, and may have different calling syntax altogether. After obtaining a display, you initialize it, set the preferred configuration, and create a surface with a back buffer you can draw into.

Obtaining a display and configuring it with EGL..

```

// Obtain an EGL display object.
EGLDisplay display = eglGetDisplay(GetDC(hWnd));
if (display == EGL_NO_DISPLAY)
{
    return EGL_FALSE;
}

// Initialize the display
if (!eglInitialize(display, &majorVersion, &minorVersion))
{
    return EGL_FALSE;
}

// Obtain the display configs
if (!eglGetConfigs(display, NULL, 0, &numConfigs))
{
    return EGL_FALSE;
}

// Choose the display config
if (!eglChooseConfig(display, attribList, &config, 1, &numConfigs))
{
    return EGL_FALSE;
}

// Create a surface
surface = eglCreateWindowSurface(display, config, (EGLNativeWindowType)hWnd,
NULL);
if (surface == EGL_NO_SURFACE)
{
    return EGL_FALSE;
}

```

In Direct3D, a UWP app's main window is represented by the [CoreWindow](#) object, which can be obtained from the app object by calling [CoreWindow::GetForCurrentThread](#) as part of the initialization process of the "view provider" you construct for Direct3D. (If you are using Direct3D-XAML interop, you use the XAML framework's view provider.) The process for creating a Direct3D view provider is covered in [How to set up your app to display a view](#).

Obtaining a CoreWindow for Direct3D.

syntax

```
CoreWindow::GetForCurrentThread();
```

Once the [CoreWindow](#) reference is obtained, the window must be activated, which executes the **Run** method of your main object and begins window event processing. After that, create an [ID3D11Device1](#) and an [ID3D11DeviceContext1](#), and use them to get

the underlying [IDXGIDevice1](#) and [IDXGIAdapter](#) so you can obtain an [IDXGIFactory2](#) object to create a swap chain resource based on your [DXGI_SWAP_CHAIN_DESC1](#) configuration.

Configuring and setting the DXGI swap chain on the CoreWindow for Direct3D.

syntax

```
// Called when the CoreWindow object is created (or re-created).
void SimpleDirect3DApp::SetWindow(CoreWindow^ window)
{
    // Register event handlers with the CoreWindow object.
    // ...

    // Obtain your ID3D11Device1 and ID3D11DeviceContext1 objects
    // In this example, m_d3dDevice contains the scoped ID3D11Device1 object
    // ...

    ComPtr<IDXGIDevice1> dxgiDevice;
    // Get the underlying DXGI device of the Direct3D device.
    m_d3dDevice.As(&dxgiDevice);

    ComPtr<IDXGIAdapter> dxgiAdapter;
    dxgiDevice->GetAdapter(&dxgiAdapter);

    ComPtr<IDXGIFactory2> dxgiFactory;
    dxgiAdapter->GetParent(
        __uuidof(IDXGIFactory2),
        &dxgiFactory);

    DXGI_SWAP_CHAIN_DESC1 swapChainDesc = {0};
    swapChainDesc.Width = static_cast<UINT>(m_d3dRenderTargetSize.Width); //
    Match the size of the window.
    swapChainDesc.Height = static_cast<UINT>(m_d3dRenderTargetSize.Height);
    swapChainDesc.Format = DXGI_FORMAT_B8G8R8A8_UNORM; // This is the most
    common swap chain format.
    swapChainDesc.Stereo = false;
    swapChainDesc.SampleDesc.Count = 1; // Don't use multi-sampling.
    swapChainDesc.SampleDesc.Quality = 0;
    swapChainDesc.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
    swapChainDesc.BufferCount = 2; // Use double-buffering to minimize
    latency.
    swapChainDesc.SwapEffect = DXGI_SWAP_EFFECT_FLIP_SEQUENTIAL; // All UWP
    apps must use this SwapEffect.
    swapChainDesc.Flags = 0;

    // ...

    Windows::UI::Core::CoreWindow^ window = m_window.Get();
    dxgiFactory->CreateSwapChainForCoreWindow(
        m_d3dDevice.Get(),
        reinterpret_cast<IUnknown*>(window),
        &swapChainDesc,
```



```
    nullptr, // Allow on all displays.  
    &m_swapChainCoreWindow);  
}
```

Call the [IDXGISwapChain1::Present1](#) method after you prepare a frame in order to display it.

Note that in Direct3D 11, there isn't an abstraction identical to EGLSurface. (There is [IDXGISurface1](#), but it is used differently.) The closest conceptual approximation is the [ID3D11RenderTargetView](#) object that we use to assign a texture ([ID3D11Texture2D](#)) as the back buffer that our shader pipeline will draw into.

Setting up the back buffer for the swap chain in Direct3D 11

syntax

```
ComPtr<ID3D11RenderTargetView>    m_d3dRenderTargetViewWin; // scoped to  
renderer object  
  
// ...  
  
ComPtr<ID3D11Texture2D> backBuffer2;  
  
m_swapChainCoreWindow->GetBuffer(0, IID_PPV_ARGS(&backBuffer2));  
  
m_d3dDevice->CreateRenderTargetView(  
    backBuffer2.Get(),  
    nullptr,  
    &m_d3dRenderTargetViewWin);
```

A good practice is to call this code whenever the window is created or changes size.

During rendering, set the render target view with

[ID3D11DeviceContext1::OMSetRenderTargets](#) before setting up any other subresources like vertex buffers or shaders.

syntax

```
// Set the render target for the draw operation.  
m_d3dContext->OMSetRenderTargets(  
    1,  
    d3dRenderTargetView.GetAddressOf(),  
    nullptr);
```

Creating a rendering context

In EGL 1.4, a "display" represents a set of window resources. Typically, you configure a "surface" for the display by supplying a set of attributes to the display object and getting a surface in return. You create a context for displaying the contents of the surface by creating that context and binding it to the surface and the display.

The call flow usually looks similar to this:

- Call `eglGetDisplay` with the handle to a display or window resource and obtain a display object.
- Initialize the display with `eglInitialize`.
- Obtain the available display configuration and select one with `eglGetConfigs` and `eglChooseConfig`.
- Create a window surface with `eglCreateWindowSurface`.
- Create a display context for drawing with `eglCreateContext`.
- Bind the display context to the display and the surface with `eglMakeCurrent`.

In the previous section, we created the `EGLDisplay` and the `EGLSurface`, and now we use the `EGLDisplay` to create a context and associate that context with the display, using the configured `EGLSurface` to parameterize the output.

Obtaining a rendering context with EGL 1.4

C++

```
// Configure your EGLDisplay and obtain an EGLSurface here ...
// ...

// Create a drawing context from the EGLDisplay
context = eglCreateContext(display, config, EGL_NO_CONTEXT, contextAttribs);
if (context == EGL_NO_CONTEXT)
{
    return EGL_FALSE;
}

// Make the context current
if (!eglMakeCurrent(display, surface, surface, context))
{
    return EGL_FALSE;
}
```

A rendering context in Direct3D 11 is represented by an [ID3D11Device1](#) object, which represents the adapter and allows you to create Direct3D resources such as buffers and shaders; and by the [ID3D11DeviceContext1](#) object, which allows you to manage the graphics pipeline and execute the shaders.

Be aware of Direct3D feature levels! These are used to support older Direct3D hardware platforms, from DirectX 9.1 to DirectX 11. Many platforms that use low power graphics hardware, such as tablets, only have access to DirectX 9.1 features, and older supported graphics hardware could be from 9.1 through 11.

Creating a rendering context with DXGI and Direct3D

C++

```
// ...

UINT creationFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;
ComPtr<IDXGIDevice> dxgiDevice;

D3D_FEATURE_LEVEL featureLevels[] =
{
    D3D_FEATURE_LEVEL_11_1,
    D3D_FEATURE_LEVEL_11_0,
    D3D_FEATURE_LEVEL_10_1,
    D3D_FEATURE_LEVEL_10_0,
    D3D_FEATURE_LEVEL_9_3,
    D3D_FEATURE_LEVEL_9_2,
    D3D_FEATURE_LEVEL_9_1
};

// Create the Direct3D 11 API device object and a corresponding context.
ComPtr<ID3D11Device> device;
ComPtr<ID3D11DeviceContext> d3dContext;

D3D11CreateDevice(
    nullptr, // Specify nullptr to use the default adapter.
    D3D_DRIVER_TYPE_HARDWARE,
    nullptr,
    creationFlags, // Set debug and Direct2D compatibility flags.
    featureLevels, // List of feature levels this app can support.
    ARRAYSIZE(featureLevels),
    D3D11_SDK_VERSION, // Always set this to D3D11_SDK_VERSION for UWP apps.
    &device, // Returns the Direct3D device created.
    &m_featureLevel, // Returns feature level of device created.
    &d3dContext // Returns the device immediate context.
);
```

Drawing into a texture or pixmap resource

To draw into a texture with OpenGL ES 2.0, configure a pixel buffer, or PBuffer. After you successfully a configure and create an EGLSurface for it you can supply it with a rendering context and execute the shader pipeline to draw into the texture.

Draw into a pixel buffer with OpenGL ES 2.0

syntax

```
// Create a pixel buffer surface to draw into
EGLConfig pBufConfig;
EGLint totalpBufAttrs;

const EGLint pBufConfigAttrs[] =
{
    // Configure the pBuffer here...
};

eglChooseConfig(eglDisplay, pBufConfigAttrs, &pBufConfig, 1,
&totalpBufAttrs);
EGLSurface pBuffer = eglCreatePbufferSurface(eglDisplay, pBufConfig,
EGL_TEXTURE_RGBA);
```

In Direct3D 11, you create an [ID3D11Texture2D](#) resource and make it a render target. Configure the render target using [D3D11_RENDER_TARGET_VIEW_DESC](#). When you call the [ID3D11DeviceContext::Draw](#) method (or a similar Draw* operation on the device context) using this render target, the results are drawn into a texture.

Draw into a texture with Direct3D 11

syntax

```
ComPtr<ID3D11Texture2D> renderTarget1;

D3D11_RENDER_TARGET_VIEW_DESC renderTargetDesc = {0};
// Configure renderTargetDesc here ...

m_d3dDevice->CreateRenderTargetView(
    renderTarget1.Get(),
    nullptr,
    &m_d3dRenderTargetViewWin);

// Later, in your render loop...

// Set the render target for the draw operation.
m_d3dContext->OMSetRenderTargets(
    1,
    d3dRenderTargetView.GetAddressOf(),
    nullptr);
```

This texture can be passed to a shader if it is associated with an [ID3D11ShaderResourceView](#).

Drawing to the screen

Once you have used your `EGLContext` to configure your buffers and update your data, you run the shaders bound to it and draw the results to the back buffer with `glDrawElements`. You display the back buffer by calling `eglSwapBuffers`.

Open GL ES 2.0: Drawing to the screen.

syntax

```
glDrawElements(GL_TRIANGLES, renderer->numIndices, GL_UNSIGNED_INT, 0);

eglSwapBuffers(drawContext->eglDisplay, drawContext->eglSurface);
```

In Direct3D 11, you configure your buffers and bind shaders with your `IDXGISwapChain::Present1`. Then you call one of the `ID3D11DeviceContext1::Draw*` methods to run the shaders and draw the results to a render target configured as the back buffer for the swap chain. After that, you simply present the back buffer to the display by calling `IDXGISwapChain::Present1`.

Direct3D 11: Drawing to the screen.

syntax

```
m_d3dContext->DrawIndexed(
    m_indexCount,
    0,
    0);

// ...

m_swapChainCoreWindow->Present1(1, 0, &parameters);
```

Releasing graphics resources

In EGL, you release the window resources by passing the `EGLDisplay` to `eglTerminate`.

Terminating a display with EGL 1.4

C++

```
EGLBoolean eglTerminate(eglDisplay);
```

In a UWP app, you can close the CoreWindow with [CoreWindow::Close](#), although this can only be used for secondary UI windows. The primary UI thread and its associated CoreWindow cannot be closed; rather, they are expired by the operating system. However, when a secondary CoreWindow is closed, the [CoreWindow::Closed](#) event is raised.

API Reference mapping for EGL to Direct3D 11

 Expand table

EGL API	Similar Direct3D 11 API or behavior
eglBindAPI	N/A.
eglBindTexImage	Call ID3D11Device::CreateTexture2D to set a 2D texture.
eglChooseConfig	Direct3D does not supply a set of default frame buffer configurations. The swap chain's configuration
eglCopyBuffers	To copy a buffer data, call ID3D11DeviceContext::CopyStructureCount . To copy a resource, call ID3D11DeviceContext::CopyResource .
eglCreateContext	Create a Direct3D device context by calling D3D11CreateDevice , which returns both a handle to a Direct3D device and a default Direct3D immediate context (ID3D11DeviceContext1 object). You can also create a Direct3D deferred context by calling ID3D11Device2::CreateDeferredContext on the returned ID3D11Device1 object.
eglCreatePbufferFromClientBuffer	All buffers are read and written as a Direct3D subresource, such as an ID3D11Texture2D . Copy from one to another compatible subresource type with a methods such as ID3D11DeviceContext1::CopyResource .
eglCreatePbufferSurface	To create a Direct3D device with no swap chain, call the static D3D11CreateDevice method. For a Direct3D render target view, call ID3D11Device::CreateRenderTargetView .
eglCreatePixmapSurface	To create a Direct3D device with no swap chain, call the static D3D11CreateDevice method. For a Direct3D render target view, call ID3D11Device::CreateRenderTargetView .
eglCreateWindowSurface	Obtain an IDXGISwapChain1 (for the display buffers) and an ID3D11Device1 (a virtual interface for the graphics device and its resources). Use the ID3D11Device1 to define an

EGL API	Similar Direct3D 11 API or behavior
	ID3D11RenderTargetView that you can use to create the frame buffer you supply to the IDXGISwapChain1 .
<code>eglDestroyContext</code>	N/A. Use ID3D11DeviceContext::DiscardView1 to get rid of a render target view. To close the parent ID3D11DeviceContext1 , set the instance to null and wait for the platform to reclaim its resources. You cannot destroy the device context directly.
<code>eglDestroySurface</code>	N/A. Graphics resources are cleaned up when the UWP app's <code>CoreWindow</code> is closed by the platform.
<code>eglGetCurrentDisplay</code>	Call CoreWindow::GetForCurrentThread to get a reference to the current main app window.
<code>eglGetCurrentSurface</code>	This is the current ID3D11RenderTargetView . Typically, this is scoped to your renderer object.
<code>eglGetError</code>	Errors are obtained as HRESULTs returned by most methods on DirectX interfaces. If the method does not return an HRESULT, call GetLastError . To convert a system error into an HRESULT value, use the HRESULT_FROM_WIN32 macro.
<code>eglInitialize</code>	Call CoreWindow::GetForCurrentThread to get a reference to the current main app window.
<code>eglMakeCurrent</code>	Set a render target for drawing on the current context with ID3D11DeviceContext1::OMSetRenderTargets .
<code>eglQueryContext</code>	N/A. However, you may acquire rendering targets from an ID3D11Device1 instance, as well as some configuration data. (See the link for the list of available methods.)
<code>eglQuerySurface</code>	N/A. However, you may acquire data about viewports and the current graphics hardware from methods on an ID3D11Device1 instance. (See the link for the list of available methods.)
<code>eglReleaseTexImage</code>	N/A.
<code>eglReleaseThread</code>	For general GPU multithreading, read Multithreading .
<code>eglSurfaceAttrib</code>	Use D3D11_RENDER_TARGET_VIEW_DESC to configure a Direct3D render target view,
<code>eglSwapBuffers</code>	Use IDXGISwapChain1::Present1 .
<code>eglSwapInterval</code>	See IDXGISwapChain1 .
<code>eglTerminate</code>	The <code>CoreWindow</code> used to display the output of the graphics

EGL API	Similar Direct3D 11 API or behavior
	pipeline is managed by the operating system.
eglWaitClient	For shared surfaces, use IDXGICkeyedMutex. For general GPU multithreading, read Multithreading .
eglWaitGL	For shared surfaces, use IDXGICkeyedMutex. For general GPU multithreading, read Multithreading .
eglWaitNative	For shared surfaces, use IDXGICkeyedMutex. For general GPU multithreading, read Multithreading .

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Compare OpenGL ES 2.0 buffers, uniforms, and vertex attributes to Direct3D

Article • 10/20/2022

Important APIs

- [ID3D11Device1::CreateBuffer](#)
- [ID3D11Device1::CreateInputLayout](#)
- [ID3D11DeviceContext1::IASetInputLayout](#)

During the process of porting to Direct3D 11 from OpenGL ES 2.0, you must change the syntax and API behavior for passing data between the app and the shader programs.

In OpenGL ES 2.0, data is passed to and from shader programs in four ways: as uniforms for constant data, as attributes for vertex data, as buffer objects for other resource data (such as textures). In Direct3D 11, these roughly map to constant buffers, vertex buffers, and subresources. Despite the superficial commonality, they are handled quite different in usage.

Here's the basic mapping.

[Expand table](#)

OpenGL ES 2.0	Direct3D 11
uniform	constant buffer (cbuffer) field.
attribute	vertex buffer element field, designated by an input layout and marked with a specific HLSL semantic.
buffer object	buffer; See D3D11_SUBRESOURCE_DATA and D3D11_BUFFER_DESC and for a general-use buffer definitions.
frame buffer object (FBO)	render target(s); See ID3D11RenderTargetView with ID3D11Texture2D .
back buffer	swap chain with "back buffer" surface; See IDXGISwapChain1 with attached IDXGISurface1 .

Port buffers

In OpenGL ES 2.0, the process for creating and binding any kind of buffer generally follows this pattern

- Call `glGenBuffers` to generate one or more buffers and return the handles to them.
- Call `glBindBuffer` to define the layout of a buffer, such as `GL_ELEMENT_ARRAY_BUFFER`.
- Call `glBufferData` to populate the buffer with specific data (such as vertex structures, index data, or color data) in a specific layout.

The most common kind of buffer is the vertex buffer, which minimally contains the positions of the vertices in some coordinate system. In typical use, a vertex is represented by a structure that contains the position coordinates, a normal vector to the vertex position, a tangent vector to the vertex position, and texture lookup (uv) coordinates. The buffer contains a contiguous list of these vertices, in some order (like a triangle list, or strip, or fan), and which collectively represent the visible polygons in your scene. (In Direct3D 11 as well as OpenGL ES 2.0 it is inefficient to have multiple vertex buffers per draw call.)

Here's an example a vertex buffer and an index buffer created with OpenGL ES 2.0:

OpenGL ES 2.0: Creating and populating a vertex buffer and an index buffer.

syntax

```
glGenBuffers(1, &renderer->vertexBuffer);
glBindBuffer(GL_ARRAY_BUFFER, renderer->vertexBuffer);
glBufferData(GL_ARRAY_BUFFER, sizeof(Vertex) * CUBE_VERTICES, renderer->vertices, GL_STATIC_DRAW);

glGenBuffers(1, &renderer->indexBuffer);
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, renderer->indexBuffer);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(int) * CUBE_INDICES, renderer->vertexIndices, GL_STATIC_DRAW);
```

Other buffers include pixel buffers and maps, like textures. The shader pipeline can render into texture buffers (pixmap) or render buffer objects and use those buffers in future shader passes. In the simplest case, the call flow is:

- Call `glGenFramebuffers` to generate a frame buffer object.
- Call `glBindFramebuffer` to bind the frame buffer object for writing.
- Call `glFramebufferTexture2D` to draw into a specified texture map.

In Direct3D 11, buffer data elements are considered "subresources," and can range from individual vertex data elements to MIP-map textures.

- Populate a [D3D11_SUBRESOURCE_DATA](#) structure with the configuration for a buffer data element.
- Populate a [D3D11_BUFFER_DESC](#) structure with the size of the individual elements in the buffer as well as the buffer type.
- Call [ID3D11Device1::CreateBuffer](#) with these two structures.

Direct3D 11: Creating and populating a vertex buffer and an index buffer.

syntax

```
D3D11_SUBRESOURCE_DATA vertexBufferData = {0};
vertexBufferData.pSysMem = cubeVertices;
vertexBufferData.SysMemPitch = 0;
vertexBufferData.SysMemSlicePitch = 0;
CD3D11_BUFFER_DESC vertexBufferDesc(sizeof(cubeVertices),
D3D11_BIND_VERTEX_BUFFER);

m_d3dDevice->CreateBuffer(
    &vertexBufferDesc,
    &vertexBufferData,
    &m_vertexBuffer);

m_indexCount = ARRAYSIZE(cubeIndices);

D3D11_SUBRESOURCE_DATA indexBufferData = {0};
indexBufferData.pSysMem = cubeIndices;
indexBufferData.SysMemPitch = 0;
indexBufferData.SysMemSlicePitch = 0;
CD3D11_BUFFER_DESC indexBufferDesc(sizeof(cubeIndices),
D3D11_BIND_INDEX_BUFFER);

m_d3dDevice->CreateBuffer(
    &indexBufferDesc,
    &indexBufferData,
    &m_indexBuffer);
```

Writable pixel buffers or maps, such as a frame buffer, can be created as [ID3D11Texture2D](#) objects. These can be bound as resources to an [ID3D11RenderTargetView](#) or [ID3D11ShaderResourceView](#), which, once drawn into, can be displayed with the associated swap chain or passed to a shader, respectively.

Direct3D 11: Creating a frame buffer object.

syntax

```
ComPtr<ID3D11RenderTargetView> m_d3dRenderTargetViewWin;
// ...
ComPtr<ID3D11Texture2D> frameBuffer;
```

```
m_swapChainCoreWindow->GetBuffer(0, IID_PPV_ARGS(&frameBuffer));  
m_d3dDevice->CreateRenderTargetView(  
    frameBuffer.Get(),  
    nullptr,  
    &m_d3dRenderTargetViewWin);
```

Change uniforms and uniform buffer objects to Direct3D constant buffers

In Open GL ES 2.0, uniforms are the mechanism to supply constant data to individual shader programs. This data cannot be altered by the shaders.

Setting a uniform typically involves providing one of the glUniform* methods with the upload location in the GPU along with a pointer to the data in app memory. After the glUniform* method executes, the uniform data is in the GPU memory and accessible by the shaders that have declared that uniform. You are expected to ensure that the data is packed in such a way that the shader can interpret it based on the uniform declaration in the shader (by using compatible types).

OpenGL ES 2.0 Creating a uniform and uploading data to it

syntax

```
renderer->mvpLoc = glGetUniformLocation(renderer->programObject,  
    "u_mvpMatrix");  
  
// ...  
  
glUniformMatrix4fv(renderer->mvpLoc, 1, GL_FALSE, (GLfloat*) &renderer->  
    mvpMatrix.m[0][0]);
```

In a shader's GLSL, the corresponding uniform declaration looks like this:

Open GL ES 2.0: GLSL uniform declaration

syntax

```
uniform mat4 u_mvpMatrix;
```

Direct3D designates uniform data as "constant buffers," which, like uniforms, contain constant data provided to individual shaders. As with uniform buffers, it is important to pack the constant buffer data in memory identically to the way the shader expects to interpret it. Using DirectXMath types (such as [XMFLOAT4](#)) instead of platform types (such as **float*** or **float[4]**) guarantees proper data element alignment.

Constant buffers must have an associated GPU register used to reference that data on the GPU. The data is packed into the register location as indicated by the layout of the buffer.

Direct3D 11: Creating a constant buffer and uploading data to it

syntax

```
struct ModelViewProjectionConstantBuffer
{
    DirectX::XMFLLOAT4X4 mvp;
};

// ...

ModelViewProjectionConstantBuffer m_constantBufferData;

// ...

XMStoreFloat4x4(&m_constantBufferData.mvp, mvpMatrix);

CD3D11_BUFFER_DESC
constantBufferDesc(sizeof(ModelViewProjectionConstantBuffer),
D3D11_BIND_CONSTANT_BUFFER);
m_d3dDevice->CreateBuffer(
    &constantBufferDesc,
    nullptr,
    &m_constantBuffer);
```

In a shader's HLSL, the corresponding constant buffer declaration looks like this:

Direct3D 11: Constant buffer HLSL declaration

syntax

```
cbuffer ModelViewProjectionConstantBuffer : register(b0)
{
    matrix mvp;
};
```

Note that a register must be declared for each constant buffer. Different Direct3D feature levels have different maximum available registers, so do not exceed the maximum number for the lowest feature level you are targeting.

Port vertex attributes to a Direct3D input layouts and HLSL semantics

Since vertex data can be modified by the shader pipeline, OpenGL ES 2.0 requires that you specify them as "attributes" instead of "uniforms". (This has changed in later versions of OpenGL and GLSL.) Vertex-specific data such the vertex position, normals, tangents, and color values are supplied to the shaders as attribute values. These attribute values correspond to specific offsets for each element in the vertex data; for example, the first attribute could point to the position component of an individual vertex, and the second to the normal, and so on.

The basic process for moving the vertex buffer data from main memory to the GPU looks like this:

- Upload the vertex data with `glBindBuffer`.
- Get the location of the attributes on the GPU with `glGetAttribLocation`. Call it for each attribute in the vertex data element.
- Call `glVertexAttribPointer` to provide set the correct attribute size and offset inside an individual vertex data element. Do this for each attribute.
- Enable the vertex data layout information with `glEnableVertexAttribArray`.

OpenGL ES 2.0: Uploading vertex buffer data to the shader attribute

syntax

```
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, renderer->vertexBuffer);
loc = glGetAttribLocation(renderer->programObject, "a_position");

glVertexAttribPointer(loc, 3, GL_FLOAT, GL_FALSE,
    sizeof(Vertex), 0);
loc = glGetAttribLocation(renderer->programObject, "a_color");
glEnableVertexAttribArray(loc);

glVertexAttribPointer(loc, 4, GL_FLOAT, GL_FALSE,
    sizeof(Vertex), (GLvoid*) (sizeof(float) * 3));
glEnableVertexAttribArray(loc);
```

Now, in your vertex shader, you declare attributes with the same names you defined in your call to `glGetAttribLocation`.

OpenGL ES 2.0: Declaring an attribute in GLSL

syntax

```
attribute vec4 a_position;
attribute vec4 a_color;
```

In some ways, the same process holds for Direct3D. Instead of attributes, vertex data is provided in input buffers, which include vertex buffers and the corresponding index

buffers. However, since Direct3D does not have the "attribute" declaration, you must specify an input layout which declares the individual component of the data elements in the vertex buffer and the HLSL semantics that indicate where and how those components are to be interpreted by the vertex shader. HLSL semantics require that you define the usage of each component with a specific string that informs the shader engine as to its purpose. For example, vertex position data is marked as POSITION, normal data is marked as NORMAL, and vertex color data is marked as COLOR. (Other shader stages also require specific semantics, and those semantics have different interpretations based on the shader stage.) For more info on HLSL semantics, read [Port your shader pipeline](#) and [HLSL Semantics](#).

Collectively, the process of setting the vertex and index buffers, and setting the input layout is called the "Input Assembly" (IA) stage of the Direct3D graphics pipeline.

Direct3D 11: Configuring the input assembly stage

syntax

```
// Set up the IA stage corresponding to the current draw operation.
UINT stride = sizeof(VertexPositionColor);
UINT offset = 0;
m_d3dContext->IASetVertexBuffers(
    0,
    1,
    m_vertexBuffer.GetAddressOf(),
    &stride,
    &offset);

m_d3dContext->IASetIndexBuffer(
    m_indexBuffer.Get(),
    DXGI_FORMAT_R16_UINT,
    0);

m_d3dContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
m_d3dContext->IASetInputLayout(m_inputLayout.Get());
```

An input layout is declared and associated with a vertex shader by declaring the format of the vertex data element and the semantic used for each component. The vertex element data layout described in the D3D11_INPUT_ELEMENT_DESC you create must correspond to the layout of the corresponding structure. Here, you create a layout for vertex data that has two components:

- A vertex position coordinate, represented in main memory as an XMFLOAT3, which is an aligned array of 3 32-bit floating point values for the (x, y, z) coordinates.
- A vertex color value, represented as an XMFLOAT4, which is an aligned array of 4 32-bit floating point values for the color (RGBA).

You assign a semantic for each one, as well as a format type. You then pass the description to [ID3D11Device1::CreateInputLayout](#). The input layout is used when we call [ID3D11DeviceContext1::IASetInputLayout](#) when you set up the input assembly during our render method.

Direct3D 11: Describing an input layout with specific semantics

syntax

```
ComPtr<ID3D11InputLayout> m_inputLayout;

// ...

const D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
      D3D11_INPUT_PER_VERTEX_DATA, 0 },
    { "COLOR",    0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
      D3D11_INPUT_PER_VERTEX_DATA, 0 },
};

m_d3dDevice->CreateInputLayout(
    vertexDesc,
    ARRAYSIZE(vertexDesc),
    fileData->Data,
    fileData->Length,
    &m_inputLayout);

// ...
// When we start the drawing process...

m_d3dContext->IASetInputLayout(m_inputLayout.Get());
```

Finally, you make sure that the shader can understand the input data by declaring the input. The semantics you assigned in the layout are used to select the correct locations in GPU memory.

Direct3D 11: Declaring shader input data with HLSL semantics

syntax

```
struct VertexShaderInput
{
    float3 pos : POSITION;
    float3 color : COLOR;
};
```

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Compare the OpenGL ES 2.0 shader pipeline to Direct3D

Article • 10/20/2022

Important APIs

- [Input-Assembler Stage](#)
- [Vertex-Shader Stage](#)
- [Pixel-Shader Stage](#)

Conceptually, the Direct3D 11 shader pipeline is very similar to the one in OpenGL ES 2.0. In terms of API design, however, the major components for creating and managing the shader stages are parts of two primary interfaces, [ID3D11Device1](#) and [ID3D11DeviceContext1](#). This topic attempts to map common OpenGL ES 2.0 shader pipeline API patterns to the Direct3D 11 equivalents in these interfaces.

Reviewing the Direct3D 11 shader pipeline

The shader objects are created with methods on the [ID3D11Device1](#) interface, such as [ID3D11Device1::CreateVertexShader](#) and [ID3D11Device1::CreatePixelShader](#).

The Direct3D 11 graphics pipeline is managed by instances of the [ID3D11DeviceContext1](#) interface, and has the following stages:

- [Input-Assembler Stage](#). The input-assembler stage supplies data (triangles, lines and points) to the pipeline. [ID3D11DeviceContext1](#) methods that support this stage are prefixed with "IA".
- [Vertex-Shader Stage](#) - The vertex-shader stage processes vertices, typically performing operations such as transformations, skinning, and lighting. A vertex shader always takes a single input vertex and produces a single output vertex. [ID3D11DeviceContext1](#) methods that support this stage are prefixed with "VS".
- [Stream-Output Stage](#) - The stream-output stage streams primitive data from the pipeline to memory on its way to the rasterizer. Data can be streamed out and/or passed into the rasterizer. Data streamed out to memory can be recirculated back into the pipeline as input data or read-back from the CPU. [ID3D11DeviceContext1](#) methods that support this stage are prefixed with "SO".
- [Rasterizer Stage](#) - The rasterizer clips primitives, prepares primitives for the pixel shader, and determines how to invoke pixel shaders. You can disable rasterization by telling the pipeline there is no pixel shader (set the pixel shader stage to NULL with [ID3D11DeviceContext::PSSetShader](#)), and disabling depth and stencil testing

(set DepthEnable and StencilEnable to FALSE in [D3D11_DEPTH_STENCIL_DESC](#)).

While disabled, rasterization-related pipeline counters will not update.

- [Pixel-Shader Stage](#) - The pixel-shader stage receives interpolated data for a primitive and generates per-pixel data such as color. [ID3D11DeviceContext1](#) methods that support this stage are prefixed with "PS".
- [Output-Merger Stage](#) - The output-merger stage combines various types of output data (pixel shader values, depth and stencil information) with the contents of the render target and depth/stencil buffers to generate the final pipeline result. [ID3D11DeviceContext1](#) methods that support this stage are prefixed with "OM".

(There are also stages for geometry shaders, hull shaders, tessellators, and domain shaders, but since they have no analogues in OpenGL ES 2.0, we won't discuss them here.) For a complete list of the methods for these stages, refer to the [ID3D11DeviceContext](#) and [ID3D11DeviceContext1](#) reference pages.

[ID3D11DeviceContext1](#) extends [ID3D11DeviceContext](#) for Direct3D 11.

Creating a shader

In Direct3D, shader resources are not created before compiling and loading them; rather, the resource is created when the HLSL is loaded. Therefore, there is no directly analogous function to `glCreateShader`, which creates an initialized shader resource of a specific type (such as `GL_VERTEX_SHADER` or `GL_FRAGMENT_SHADER`). Rather, shaders are created after the HLSL is loaded with specific functions like

[ID3D11Device1::CreateVertexShader](#) and [ID3D11Device1::CreatePixelShader](#), and which take the type and the compiled HLSL as parameters.

 Expand table

OpenGL ES 2.0	Direct3D 11
<code>glCreateShader</code>	Call ID3D11Device1::CreateVertexShader and ID3D11Device1::CreatePixelShader after successfully loading the compiled shader object, passing them the CSO as a buffer.

Compiling a shader

Direct3D shaders must be precompiled as Compiled Shader Object (.cso) files in Universal Windows Platform (UWP) apps and loaded using one of the Windows Runtime file APIs. (Desktop apps can compile the shaders from text files or string at run-time.)

The CSO files are built from any .hlsl files that are part of your Microsoft Visual Studio project, and retain the same names, only with a .cso file extension. Ensure that they are included with your package when you ship!

 Expand table

OpenGL ES 2.0	Direct3D 11
glCompileShader	N/A. Compile the shaders to .cso files in Visual Studio and include them in your package.
Using glGetShaderiv for compile status	N/A. See the compilation output from Visual Studio's FX Compiler (FXC) if there are errors in compilation. If compilation is successful, a corresponding CSO file is created.

Loading a shader

As noted in the section on creating a shader, Direct3D 11 creates the shader when the corresponding CSO file is loaded into a buffer and passed to one of the methods in the following table.

 Expand table

OpenGL ES 2.0	Direct3D 11
ShaderSource	Call ID3D11Device1::CreateVertexShader and ID3D11Device1::CreatePixelShader after successfully loading the compiled shader object.

Setting up the pipeline

OpenGL ES 2.0 has the "shader program" object, which contains multiple shaders for execution. Individual shaders are attached to the shader program object. However, in Direct3D 11, you work with the rendering context ([ID3D11DeviceContext1](#)) directly and create shaders on it.

 Expand table

OpenGL ES 2.0	Direct3D 11
glCreateProgram	N/A. Direct3D 11 does not use the shader program object abstraction.
glLinkProgram	N/A. Direct3D 11 does not use the shader program object abstraction.
glUseProgram	N/A. Direct3D 11 does not use the shader program object abstraction.
glGetProgramiv	Use the reference you created to ID3D11DeviceContext1 .

Create an instance of [ID3D11DeviceContext1](#) and [ID3D11Device1](#) with the static [D3D11CreateDevice](#) method.

syntax

```
Microsoft::WRL::ComPtr<ID3D11Device1>          m_d3dDevice;
Microsoft::WRL::ComPtr<ID3D11DeviceContext1>    m_d3dContext;

// ...

D3D11CreateDevice(
    nullptr, // Specify nullptr to use the default adapter.
    D3D_DRIVER_TYPE_HARDWARE,
    nullptr,
    creationFlags, // Set set debug and Direct2D compatibility flags.
    featureLevels, // List of feature levels this app can support.
    ARRAYSIZE(featureLevels),
    D3D11_SDK_VERSION, // Always set this to D3D11_SDK_VERSION for UWP apps.
    &device, // Returns the Direct3D device created.
    &m_featureLevel, // Returns feature level of device created.
    &m_d3dContext // Returns the device's immediate context.
);
```

Setting the viewport(s)

Setting a viewport in Direct3D 11 is very similar to how you set a viewport in OpenGL ES 2.0. In Direct3D 11, call [ID3D11DeviceContext::RSSetViewports](#) with a configured [CD3D11_VIEWPORT](#).

Direct3D 11: Setting a viewport.

syntax

```
CD3D11_VIEWPORT viewport(
    0.0f,
    0.0f,
```

```

        m_d3dRenderTargetSize.Width,
        m_d3dRenderTargetSize.Height
    );
    m_d3dContext->RSSetViewports(1, &viewport);

```

[Expand table](#)

OpenGL ES 2.0	Direct3D 11
glViewport	CD3D11_VIEWPORT , ID3D11DeviceContext::RSSetViewports

Configuring the vertex shaders

Configuring a vertex shader in Direct3D 11 is done when the shader is loaded. Uniforms are passed as constant buffers using [ID3D11DeviceContext1::VSSetConstantBuffers1](#).

[Expand table](#)

OpenGL ES 2.0	Direct3D 11
glAttachShader	ID3D11Device1::CreateVertexShader
glGetShaderiv, glGetShaderSource	ID3D11DeviceContext1::VSGetShader
glGetUniformfv, glGetUniformiv	ID3D11DeviceContext1::VSGetConstantBuffers1 .

Configuring the pixel shaders

Configuring a pixel shader in Direct3D 11 is done when the shader is loaded. Uniforms are passed as constant buffers using [ID3D11DeviceContext1::PSSetConstantBuffers1](#).

[Expand table](#)

OpenGL ES 2.0	Direct3D 11
glAttachShader	ID3D11Device1::CreatePixelShader
glGetShaderiv, glGetShaderSource	ID3D11DeviceContext1::PSGetShader
glGetUniformfv, glGetUniformiv	ID3D11DeviceContext1::PSGetConstantBuffers1 .

Generating the final results

When the pipeline completes, you draw the results of the shader stages into the back buffer. In Direct3D 11, just as it is with Open GL ES 2.0, this involves calling a draw command to output the results as a color map in the back buffer, and then sending that back buffer to the display.

 Expand table

OpenGL ES 2.0	Direct3D 11
glDrawElements	ID3D11DeviceContext1::Draw , ID3D11DeviceContext1::DrawIndexed (or other Draw* methods on ID3D11DeviceContext1).
eglSwapBuffers	IDXGISwapChain1::Present1

Porting GLSL to HLSL

GLSL and HLSL are not very different beyond complex type support and syntax some overall syntax. Many developers find it easiest to port between the two by aliasing common OpenGL ES 2.0 instructions and definitions to their HLSL equivalent. Note that Direct3D uses the Shader Model version to express the feature set of the HLSL supported by a graphics interface; OpenGL has a different version specification for HLSL. The following table attempts to give you some approximate idea of the shader language feature sets defined for Direct3D 11 and OpenGL ES 2.0 in the terms of the other's version.

 Expand table

Shader language	GLSL feature version	Direct3D Shader Model
Direct3D 11 HLSL	~4.30.	SM 5.0
GLSL ES for OpenGL ES 2.0	1.40. Older implementations of GLSL ES for OpenGL ES 2.0 may use 1.10 through 1.30. Check your original code with <code>glGetString(GL_SHADING_LANGUAGE_VERSION)</code> or <code>glGetString(SHADING_LANGUAGE_VERSION)</code> to determine it.	~SM 2.0

For more details of differences between the two shader languages, as well as common syntax mappings, read the [GLSL-to-HLSL reference](#).

Porting the OpenGL intrinsics to HLSL semantics

Direct3D 11 HLSL semantics are strings that, like a uniform or attribute name, are used to identify a value passed between the app and a shader program. While they can be any of a variety of possible strings, the best practice is to use a string like POSITION or COLOR that indicates the usage. You assign these semantics when you are constructing a constant buffer or buffer input layout. You can also append a number between 0 and 7 to the semantic so that you use separate registers for similar values. For example: COLOR0, COLOR1, COLOR2...

Semantics that are prefixed with "SV_" are system value semantics that are written to by your shader program; your app itself (running on the CPU) cannot modify them. Typically, these contain values that are inputs or outputs from another shader stage in the graphics pipeline, or are generated entirely by the GPU.

Additionally, SV_ semantics have different behaviors when they are used to specify input to or output from a shader stage. For example, SV_POSITION (output) contains the vertex data transformed during the vertex shader stage, and SV_POSITION (input) contains the pixel position values interpolated during rasterization.

Here are a few mappings for common OpenGL ES 2.0 shader intrinsics:

 Expand table

OpenGL system value	Use this HLSL Semantic
gl_Position	POSITION(n) for vertex buffer data. SV_POSITION provides a pixel position to the pixel shader and cannot be written by your app.
gl_Normal	NORMAL(n) for normal data provided by the vertex buffer.
gl_TexCoord[n]	TEXCOORD(n) for texture UV (ST in some OpenGL documentation) coordinate data supplied to a shader.
gl_FragColor	COLOR(n) for RGBA color data supplied to a shader. Note that it is treated identically to coordinate data; the semantic simply helps you identify that it is color data.

OpenGL system value	Use this HLSL Semantic
gl_FragData[n]	SV_Target[n] for writing from a pixel shader to a target texture or other pixel buffer.

The method by which you code for semantics is not the same as using intrinsics in OpenGL ES 2.0. In OpenGL, you can access many of the intrinsics directly without any configuration or declaration; in Direct3D, you must declare a field in a specific constant buffer to use a particular semantic, or you declare it as the return value for a shader's **main()** method.

Here's an example of a semantic used in a constant buffer definition:

C++

```
struct VertexShaderInput
{
    float3 pos : POSITION;
    float3 color : COLOR0;
};

// The position is interpolated to the pixel value by the system. The per-vertex color data is also interpolated and passed through the pixel shader.
struct PixelShaderInput
{
    float4 pos : SV_POSITION;
    float3 color : COLOR0;
};
```

This code defines a pair of simple constant buffers

And here's an example of a semantic used to define the value returned by a fragment shader:

C++

```
// A pass-through for the (interpolated) color data.
float4 main(PixelShaderInput input) : SV_TARGET
{
    return float4(input.color,1.0f);
}
```

In this case, SV_TARGET is the location of the render target that the pixel color (defined as a vector with four float values) is written to when the shader completes execution.

For more details on the use of semantics with Direct3D, read [HLSL Semantics](#).

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Port a simple OpenGL ES 2.0 renderer to Direct3D 11

Article • 10/20/2022

For this porting exercise, we'll start with the basics: bringing a simple renderer for a spinning, vertex-shaded cube from OpenGL ES 2.0 into Direct3D, such that it matches the DirectX 11 App (Universal Windows) template from Visual Studio 2015. As we walk through this port process, you will learn the following:

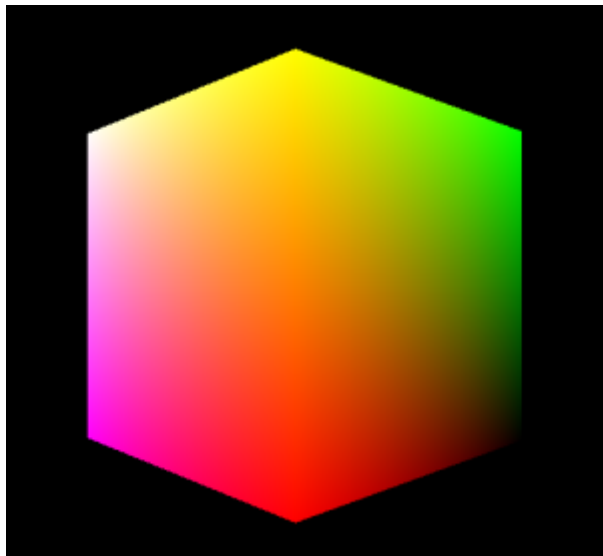
- How to port a simple set of vertex buffers to Direct3D input buffers
- How to port uniforms and attributes to constant buffers
- How to configure Direct3D shader objects
- How basic HLSL semantics are used in Direct3D shader development
- How to port very simple GLSL to HLSL

This topic starts after you have created a new DirectX 11 project. To learn how to create a new DirectX 11 project, read [Create a new DirectX 11 project for Universal Windows Platform \(UWP\)](#).

The project created from either of these links has all the code for the [Direct3D](#) infrastructure prepared, and you can immediately start into the process of porting your renderer from Open GL ES 2.0 to Direct3D 11.

This topic walks two code paths that perform the same basic graphics task: display a rotating vertex-shaded cube in a window. In both cases, the code covers the following process:

1. Creating a cube mesh from hardcoded data. This mesh is represented as a list of vertices, with each vertex possessing a position, a normal vector, and a color vector. This mesh is put into a vertex buffer for the shading pipeline to process.
2. Creating shader objects to process the cube mesh. There are two shaders: a vertex shader that processes the vertices for rasterization, and a fragment (pixel) shader that colors the individual pixels of the cube after rasterization. These pixels are written into a render target for display.
3. Forming the shading language that is used for vertex and pixel processing in the vertex and fragment shaders, respectively.
4. Displaying the rendered cube on the screen.



Upon completing this walkthrough, you should be familiar with the following basic differences between Open GL ES 2.0 and Direct3D 11:

- The representation of vertex buffers and vertex data.
- The process of creating and configuring shaders.
- Shading languages, and the inputs and outputs to shader objects.
- Screen drawing behaviors.

In this walkthrough, we refer to a simple and generic OpenGL renderer structure, which is defined like this:

syntax

```
typedef struct
{
    GLfloat pos[3];
    GLfloat rgba[4];
} Vertex;

typedef struct
{
    // Integer handle to the shader program object.
    GLuint programObject;

    // The vertex and index buffers
    GLuint vertexBuffer;
    GLuint indexBuffer;

    // Handle to the location of model-view-projection matrix uniform
    GLint  mvpLoc;

    // Vertex and index data
    Vertex *vertices;
    GLuint *vertexIndices;
    int     numIndices;
```

```
// Rotation angle used for animation
GLfloat  angle;

GLfloat  mvpMatrix[4][4]; // the model-view-projection matrix itself
} Renderer;
```

This structure has one instance and contains all the necessary components for rendering a very simple vertex-shaded mesh.

Note Any OpenGL ES 2.0 code in this topic is based on the Windows API implementation provided by the Khronos Group, and uses Windows C programming syntax.

What you need to know

Technologies

- [Microsoft Visual C++](#)
- OpenGL ES 2.0

Prerequisites

- Optional. Review [Port EGL code to DXGI and Direct3D](#). Read this topic to better understand the graphics interface provided by DirectX.

Steps

 Expand table

Topic	Description
Port the shader objects	When porting the simple renderer from OpenGL ES 2.0, the first step is to set up the equivalent vertex and fragment shader objects in Direct3D 11, and to make sure that the main program can communicate with the shader objects after they are compiled.
Port the vertex buffers and data	In this step, you'll define the vertex buffers that will contain your meshes and the index buffers